

Code-Layout, Readability and Reusability

Date: 30 September 2026 **Team ID:** 6ab3e57da238dc999eb3d44d **Project Name:** EduGenie – AI-Powered Educational Assistant
Maximum Marks: 5 Marks

Code Layout Checklist

S.No	Code Quality Parameter	Status	Evidence / Remarks
1	Consistent indentation	Yes	Python files use consistent indentation and readable function blocks.
2	Proper file structure	Yes	Frontend, backend, feature modules, static assets and documentation are separated.
3	Meaningful variable names	Yes	Names such as payload, topic, ai_service and model_name describe their purpose.
4	Function / method names	Yes	Functions such as answer_question, explain_topic, summarize_text and generate_quiz are descriptive.
5	Code comments	Partial	Comments are limited; the code relies mainly on clear module/function structure.
6	Modular design	Yes	Five workflows are separate modules and share one AI service.
7	No redundant code	Yes	Provider configuration is centralized rather than duplicated across feature modules.
8	Error handling	Yes	Runtime errors and malformed quiz JSON are handled explicitly.

Reusable Components / Modules

Component	Technology	Where Reused	Reusability
AIService	Python / google-genai	All five learning modules	High
TextRequest / TopicRequest	Pydantic	Multiple FastAPI routes	High
Feature modules	Python	One module per learning workflow	High
Frontend task/result flow	HTML/JS	All tools in one interface	Medium

Overall assessment

The strongest maintainability feature is separation of concerns: API routing, provider configuration, feature prompts and presentation are kept in different files. A future refactor can add tests, authentication, persistence and structured response schemas without replacing the core architecture.